A Scheduling Algorithm for Shared Sensor and Actuator Networks

Claudio M. de Farias, Luci Pirmez, Flávia C. Delicato
PPGI-iNCE/DCC-IM/Universidade Federal do Rio de Janeiro
Rio de Janeiro, Brazil

Wei Li, Albert Y. Zomaya
School of Information Technologies
The University of Sydney
Sydney, Australia

José N de Souza
Group of Computer Networks, Software Engineering and Systems
Universidade Federal do Ceará
Ceará, Brazil

***Abstract*—Shared Sensor and Actuator Networks (SSAN) represents a new design trend of Wireless Sensor Network (WSNs) that allows the system infrastructure to be shared among multiple applications submitted by different users instead of the original application-specific network design. In this context, we propose a task scheduling algorithm that well utilizes the characteristics of applications with common tasks to improve the system energy efficiency, so as to extend the most concerned performance metric system lifetime. Our proposal is validated by both computer simulations and real sensor nodes tests.**

Keywords—shared sensor networks; wireless sensor and actuator networks, task scheduling.

I. INTRODUCTION

Recent years we have witnessed the Shared Sensor and Actuator Networks (SSAN) become a core component of cyber-physical systems for diverse applications [1]. As Wireless Sensor and Actuator Networks (WSANs) [2] design starts shifting from application-specific platforms to shared system infrastructures, a new but pressing research challenge is how to allocate limited node resources to contending applications running on such system. This issue has received notable attention from both academic and industry sides. One possible solution is to employ task scheduling algorithms, which are responsible for energy conservation as well as application managements. By tasks, we mean the non-dividable units of execution that consist of an application submitted by users. Although several efforts have been made, most of existing works concern about the case of running an application in single WSN with simple objective to achieve energy efficiency. Task scheduling algorithms [3] designed for SSANs should not only pay close attention to the energy saving by altering functional parts of sensor nodes in their lifetime, but should also fully utilize the common tasks from different applications so as to further improve the use of limited precious node resources. This is practical due to the fact that tasks from multiple applications could run only once and the collected result is able to be shared by all of them. And the common tasks, if not properly addressed by the scheduling algorithm, will consume system energy in a less efficient way by repeatedly perform them. This situation is normally to be appeared in periodic applications, e.g. a fire detection application and a heating application could share not only a temperature sensing task but also a temperature threshold evaluation task. Besides that, aperiodic applications with short execution length in the SSAN also have chance to experience the similar situation, e.g. meeting detection application in [4]. The meeting detection application requires a presence sensor and a temperature sensor to analyze if the attendees are comfortable during a scheduled meeting (if the room's temperature is satisfactory in relation to the attendance). A fire detection application and a meeting application can share a temperature sensing task and a temperature threshold evaluation task. Moreover, the aperiodic application could share common tasks with periodic applications in the system; energy efficiency can be further improved by exploiting commonality of the application interests. For instance, the meeting detection application and the fire detection application have the same interest to collect data from the temperature sensor for their completion.

To address the aforementioned issues, this paper presents an energy-efficient scheduling algorithm to perform multiple applications in SSANs. We modeled applications as DAGs (Directed Acyclic Graphs) where each node of a DAG is a task and the edges represent the dependencies between tasks. To maintain task dependencies, we combine the DAGs with common tasks into a new DAG without damage the integrity of each task graph. Once the DAGs are combined, the applications are then allocated to the SSAN based on DAG's composition [3]. The composited DAG could contain periodic applications or aperiodic applications during its execution.

The rest of this paper is organized as follows. Section II reviews the related works. Section III introduces the models used in our work. In section IV, we present our designed scheduling algorithm for SSANs. In Section V, we describe the experiments to evaluate the proposal. Section VI concludes this paper and outlines future work.

II. RELATED WORK

The SSAN approach has been recently proposed by Efstratiou et al. [5] as an extension to the traditional design of WSAN. The idea behind the approach is tried to decouple the system infrastructure from application ownerships. A framework is created to allow WSAN infrastructure to be

This work is partly supported by the Brazilian Funding Agencies CNPq and FAPERJ under grants numbers 477223/2012-5; 473851/2012-1;4781174/2010-1;309270/2009; MCT/FINEP PLATCOG 01.10.0549.00; MCT/FINEP USICONTROL 01.10.0064.00 and FAPERJ (E-26/102.961/2012).

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DO RIO DE JANEIRO. Downloaded on April 30,2025 at 20:24:08 UTC from IEEE Xplore. Restrictions apply.

shared among multiple applications, which are potentially owned by different users. By achieving this goal, WSAN infrastructure is viewed as an accessible resource, which can be dynamically re-purposed and re-programmed by different authorities, in order to support multiple applications.

There are several works in the SSANs field that address the task scheduling problem. In [6], the authors presented Utility-based Multi-application Allocation and Deployment Environment (UMADE), which is a task allocation system for distributing various applications based on QoM (quality of monitoring). QoM is a distributed quality metric for monitoring a physical phenomenon of interest that is based on the measurement accuracy. Thus, the quality of QoM depends on the monitoring performed by all nodes allocated to an application. Unlike traditional approaches that usually allocate nodes in the networks to a single application according to metrics such as the amount of resources used, delay, processing and power consumption, UMADE dynamically allocates nodes to multiple applications according to the application's QoM. An inherent property of an application is that sensing data belonging to sensors that are allocated to a same application are naturally correlated. As a consequence, the contribution of a sensor node for an application QoM is dependent of other sensor nodes allocated for the same application.

In [7], the authors propose a greedy algorithm to schedule applications onto a specific SSAN. The algorithm performs the task allocation taking into account the QoM of the applications. The applications' QoM depends on the node to which the application was allocated. Therefore, it is important that the allocation algorithm seeks to optimize the allocation among multiple applications of a SSAN to maximize the QoM. The proposed work uses the property of QoM sub-modularity. The QoM sub-modularity is due to the fact that the readings from the sensors of different nodes are often correlated. For example, once the temperature readings from different nodes in the same room are correlated with each other, the assignment of a new node in the room to perform the monitoring temperature does not produce a considerable QoM improvement. The main difference between the proposed work and [6], [7] is that we share all kinds of tasks not only sensing tasks, what leads to a greater reduction in energy consumption.

III. MODELS AND PROBLEM DEFINITION

A. System Model

Our approach is built upon the design of SenShare [8], an architecture for SSN that allows the creation of overlay WSNs on top of a physical shared infrastructure. In an overlay network each sensor node allows multiple co-located applications to use its hardware resource (we could have used another Virtual Network approach but Senshare was the only one specific to the SSAn scenario). An application can be distributed over the whole infrastructure or a selected sub-set of the physical network. SenShare provides a clear separation between the system infrastructure and the individual applications, allowing ownership and management of the infrastructure and applications by different authorities. SenShare's goal is to map applications into physical nodes to operate in the virtual environment built on the real sensor network. From the application point of view, it is assigned to run in an overlay WSN, while in the context of the SSAN, the application is allocated to a subset of selected nodes. The overlay WSNs may share real nodes in the SSAN. In this way, an application can span across the whole infrastructure or a selected sub-set of the physical network.

By adopting Senshare as the infrastructure for our study, the sink node is responsible for the formation of the overlay wireless sensor networks. Each application in our model represents an overlay WSN in SenShare. By using SenShare, we ensure that all the intermediate results of application can freely travel the nodes that form the overlay network and reach the sink node. The sink node uses the *Network I/O* component presented in [8] and our system encompasses one SSAN. The SSAN considered in this work includes a sink node and a set of heterogeneous sensor and actuator nodes that can belong to multiple physical networks. The sink node is responsible for the overlay network formation as well as executing the proposed scheduling algorithm. The sink acts as a facade to the entire system responsible for handling the dynamic arrival of applications, decomposing an application into tasks, determining the specific nodes required for processing the application, deciding how to dispatch the tasks to the selected nodes and gathering the latest information about the network execution context and sensor nodes status. The information includes, for each sensor node, its sensing capabilities, current residual energy and operation mode and geographical location. Such information is acquired by messages sent by sensor nodes whenever required from the sink (normally at the time of a successful completion of the allocated tasks of an application).

Any given SSAN is modeled as an undirected graph $G= (V, E)$, where $V = (v_1,v_2,...,v_n)$ represents the set of sensor nodes and $E = (e_1,e_2,...,e_m)$ represents the set of all possible communication links among the sensor nodes in the same WSAN. We assume that there is a SSAN G, formed by a group of overlay WSANs = $\{G_1,..,G_n\}$, deployed into the monitored area W so that all interested events of the applications can be detected by at least one sensor. All overlay networks in G have a routing path to the sink node SN.

Any sensor node can overlap the monitoring region of a set of other sensor nodes. This indicates that some regions of the monitored area W are mutually covered by multiple sensors that can perform a certain task. For any given sensor node V_i in V, i denotes the index of the sensor nodes that belongs to the SSAN. A sensor node V_i is capable of providing one or more tasks depending on their capabilities to collect/sample different types of data, for instance, temperature, light, smoke, and movement. All sensors are endowed with the same radio interface as well as communication and sensing ranges. Moreover, sensors can detect all events of interested occurred within their sensing range. We assume that all the sensors in the SSAN have a valid communication path to reach the SN. This is guaranteed by the SenShare infrastructure that manages the formation of overlay networks as presented in

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DO RIO DE JANEIRO. Downloaded on April 30,2025 at 20:24:08 UTC from IEEE Xplore. Restrictions apply.

[8]. Communication interference between sensors is not considered in this paper. In order to examine the availability of a sensor, we introduce an availability function that indicates whether a sensor can provide the required service at the specified area. The function is shown below.

$$A(t,x,y,i) = \begin{cases} 1, & \text{if a sensor is available} \\ 0, & \text{if a sensor is unavailable} \end{cases} \quad (1)$$

where t is the task that an application requires, x and y are the geographical location for the monitoring event and i is the index of the sensor. In order to evaluate the node energy consumption, we adopted the energy consumption model used in [9].

B. Application Model

We represent applications as Directed Acyclic Graphs (DAG) [3]. A DAG T = (N, A) consists of a set of vertices N representing the tasks to be executed and a set of directed edges A representing dependencies among tasks. The edge set A contains directed edges a_{ij} for each task n_i which is the parent node of the task n_j. In DAG, a task without parent tasks is called entry-task and a task without child tasks is called exit-task. We stipulate all entry tasks of a DAG are sensing tasks, meaning the tasks that represent the different sensing functions requested from the users. The communication cost is only counted when two adjacent non-communication tasks are allocated to different sensor nodes, otherwise the cost is ignored.

IV. The Proposed Algorithm

In this section, we present a task scheduling algorithm tailored for SSAN. In the SSAN scenario, multiple applications could have common tasks. Our algorithm exploits the commonality in tasks to achieve energy efficiency.

As we mentioned before, there are multiple applications running in SSAN, an obvious way to complete these applications is to schedule them one by one. The immediate issue with this approach is that it might leave the nodes in idle status for a long time and increase the overall end-to-end delay of applications, since each task is non-preemptive, which means when the task starts execution and it can not be terminated until it is completed [12]. Another potential issue with this means is to produce more intermediate result exchange between nodes, thus reducing the network lifetime. In other words, such approach does not take advantage of the fact that applications may share common tasks. An alternative approach [3] is to schedule several DAGs at the same time, transforming the selected DAGs into a single DAG. The approach proposed in [3] creates a composite graph by making: (i) the entry nodes of all DAGs as immediate successors nodes of a single dummy entry node and (ii) the exit nodes of all DAGs immediate as ancestors nodes of a single dummy exit node. These two extra dummy nodes are associated with zero computation and communication cost. Besides that, we also need to combine the DAGs with common tasks into a single DAG and distribute the composite DAG to appropriate sensor nodes. The pseudo code of the algorithm is given in Figure 1:

Input: A set of applications (DAGs), system topology, application requirements

Output: The task scheduling scheme

1. **//Formation of a new DAG**
2. Define a Final DAG (FD) composed of an Entry Node (EN) and a Final Node (FN).
3. When a new DAG A_i arrives, decomposes the application A_i into n tasks, where $\mathbf{N} = \{n_1, n_2, \ldots, n_n\}$.
4. **For** each task v_i in V do
5. For a given task $n_i \in \mathbf{N}$, verify if this task is in FD.
6. If v_i is in FD, the ancestor of n_i in A_i will point to n_i in FD and n_i in FD will point to the successor of n_i in A_i
7. If the entry node of A_i has no ancestor, it becomes EN and if the final node of A_i has no successor it becomes FN.
8. Put all the tasks of FD into a queue Q

//Sensor availability

9. **While** size(Q) >0 **do**
10. For a given task $n_i \in \mathbf{N}$, check the sensor availability of sensors by using equation (1)
11. Create a candidate sensors list CS_i and put the sensors with value 1 into the list
12. Remove the task n_i from queue Q
13. **End While**
14. **//Scheduling**
15. **For** each candidate sensors list CS_i
16. **For** each $v_j \in CS_i$ **do**
17. select the sensor with the closest value to the result generated by equation (2)
18. **End If**
19. **End For**
20. Return the optimal sensor v_j for n_i
21. **End For**
22. The latest information of nodes which perform the allocated tasks of application A_i is collected by their sink nodes and send back to application planner for next arrival application

Figure 1. the pseudo code of proposed algorithm

In the following subsections we will provide a detailed explanation of our algorithm. It encompasses 3 steps: (i) new DAG formation, (ii) sensor availability evaluation and (iii) scheduling. In the first step, the algorithm identifies and eliminates common tasks. Secondly, the nodes that can perform the required tasks are selected and in the last step tasks are assigned to the chosen sensors.

A. New DAG Formation

When a new application arrives, the sink node will analyze the composition of the application. First of all, the proposed algorithm will add the new arrival application into the existing

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DO RIO DE JANEIRO. Downloaded on April 30,2025 at 20:24:08 UTC from IEEE Xplore. Restrictions apply.

DAG. The new arrival application is placed between the dummy entry node and dummy exit node. Then, it will search in the new arrival application for tasks are common to the applications that are currently running in the system. Using the precedence, our algorithm excludes the common tasks (in this case represented as nodes of the DAG). The ancestor of the common node (task presented in more than one application) on the arriving application will point to the node that is already in the DAG and the node in the tree will point to the successor in the arriving application.

B. Sensor availability

For each task, all possible sensor nodes capable of executing the required task are identified. Due to the hardware limitation of sensor node, each sensor node is constrained by its sensing range and only the physical phenomenon within this range can be detected. As a result, those sensors are filtered; by considering locations of those available sensors, their sensing ranges and the location of requested task, and filtered sensors are put into a candidate sensor list.

C. Scheduling

As the performance of the proposed algorithm relies on task sharing to reduce energy consumption, we derive a formula to represent energy consumption on a node, and quantitatively select the best candidate from a candidate sensor list. The normalization technique is employed to generate equation (2).

$$E_{candidate} = \frac{E_{residual} - E_{s_i}}{E_{residual}} \quad (2)$$

For a given task, $E_{residual}$ represents the residual energy of the SSAN. E_{s_i} denotes the energy consumption value for any sensor in the primary sensor list and it ranges from E_{best} to ∞. We use normalization because sensor may have different batteries, and we are interested in how a sensor is positioned in relation to the network.

V. EXPERIMENTS

To evaluate the proposed algorithm performance under a realistic scenario, we implemented the algorithm in real sensor nodes and run a set of tests under the context tailored for SSANs. The goal of the tests is to evaluate the concerned system performance metrics: system lifetime and scheduling success rate.

The first experiment aimed at assessing the system lifetime, evaluating the energy consumption of our algorithm compared to a **naive approach**, a scheme for task scheduling that was created as a baseline. In such scheme, each task is allocated in a random way that considers if the node is able to execute the selected task (in terms of geographical position, required sensing interfaces and amount of energy to conclude the task). If the node randomly selected is not able to perform the task, another node will be randomly chosen. In the second experiment, we evaluated the influence of packet loss on the system lifetime. Packet loss is important since it allows us to analyze how resilient the proposed algorithm is. In the third experiment, we evaluated the scheduling success rate to determine if our algorithm is able to, besides saving energy, properly allocate tasks to meet the application requirements. Finally, we compare the performance of our algorithm with the algorithm proposed in [6].

A. Experimental Settings

The tests were conducted in the SUN SPOT platform [10], a sensor platform particularly suitable for rapid prototyping of WSNs applications. The SUN SPOT SDK environment includes Solarium, that contains a SPOT emulator that is useful for testing SPOT software and/or to create scenarios with a large number of nodes whenever the real hardware is not available. In the performed tests, we created a mix of real nodes with emulated SPOTS by using Solarium so that we can assess the system scalability.

B. Parameter Setup

In our tests, there are several parameters that need to be properly specified considering the application, system and energy models. We assume that applications arrive to the SSAN following a Poisson distribution with a mean of 10 time units. At each arrival time, it is likely that more than one application arrive, and this probability is controlled by a parameter called *multiple application arrival rate* [11] that ranges from 0 to 1 with the default value set to 0.1. The larger the value of multiple application arrival rate is, the higher is the possibility of multiple applications to occur at each arrival time. The maximum number of applications arriving to the SSAN at the same time is set to 3. Each arriving application contains 2 to 6 independent tasks. This procedure of random graphs generation was further explained in [3]. It is discussed in the literature that random graphs may not always represent real applications; however, the diversity they provide is sufficient for this group of experiments as explained in [3]. A sensor field was created with 200 nodes randomly distributed in a square area with 200m x 200 m. A single sink node was placed in the top right corner of the field. We used LQRP [12] as routing protocol to transmit data through nodes. Unless specifically stated, all the aforementioned parameter values will be used as default setting in the experiments. All our tests compare the results of the proposed algorithm with a benchmark, the **naive approach** previously described. We initially assumed that all transmissions were carried out without data loss. Further we introduced a data loss model in the simulations, in order to evaluate the impact of packet loss on the performance of the proposed algorithm. Each simulation runs for 900s at the end of which the network residual energy was obtained. All the presented results were averaged over at least 300 times; consequently, a 95% confidence interval with a 5% (or better) precision was achieved.

C. Performance Metrics

In our experiments, the **system lifetime** and **SSR** are used as the concerned performance metrics for the evaluation. System lifetime is probably the most important metric for the SSANs performance evaluation. In this paper, the main goal of the SSAN is to appropriately schedule tasks to save energy so as to extend the system lifetime. However, the energy

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DO RIO DE JANEIRO. Downloaded on April 30,2025 at 20:24:08 UTC from IEEE Xplore. Restrictions apply.

consumption metric does not fully demonstrate the superior performance of our algorithm. We also adopt two other metrics used in [1]: *(i) number of executed tasks* and *(ii) number of missed tasks*. The number of executed tasks is used to measure the total number of tasks is successfully executed in the experiment. The number of missed tasks is used to measure the total number of tasks is failed to execute due to various issues occurred in the system, such as packet loss, sensor malfunctioning, loss of connectivity and node's energy depletion [3]. For example, due to insufficient energy, a given task n_i may only be scheduled or executed for a reduced number of times within the scheduling interval. Based on these two metrics, we derive **Scheduling Success Rate (SSR)**, which is known as *number of executed tasks / (number of executed tasks + number of missed tasks)*.

D. System lifetime

The main goal of this test is to assess how long the SSAN would last under the given number of applications. Results shown in Figure 2 indicate that, with the increment in the number of applications running in the SSAN, the system lifetime is dropped. More importantly, as expected, the relationship between the number of applications and the system lifetime is not linear. This is because that more applications are used as input naturally increase the possibility of finding common tasks between them, thus the proposed algorithm is able to fully utilize the common tasks to reduce the system energy consumption by executing those tasks only once and shared the result among the required applications.

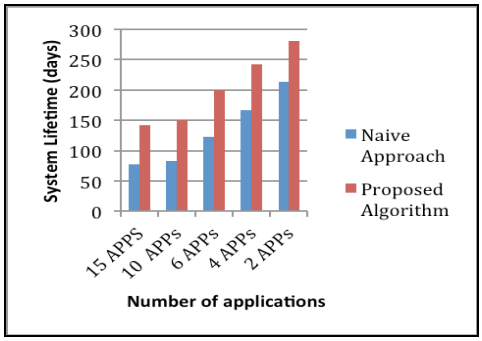


Figure 2. System lifetime with different number of applications

E. Scheduling Success Rate

As previously mentioned, SSR refers to percentage of tasks that are deployed in the system and successfully performed. If a task is missed, it might compromise all its child tasks from the same application execution and force the sink node to redeploy these tasks in the system. Otherwise, the data sent back to the user can be flawed and further leads to wrong decision/action, or even an application could be unattended. We verified the SSR by varying the number of applications in the system, as shown in Figure 3. We can observe when the number of applications increases, the SSR for both algorithms are decreased. Compared to the benchmark, the proposed algorithm can guarantee more than 80% SSR under all test conditions, but the naive approach is failed to deliver such performance due to it does not employ any technique to utilize the task sharing.

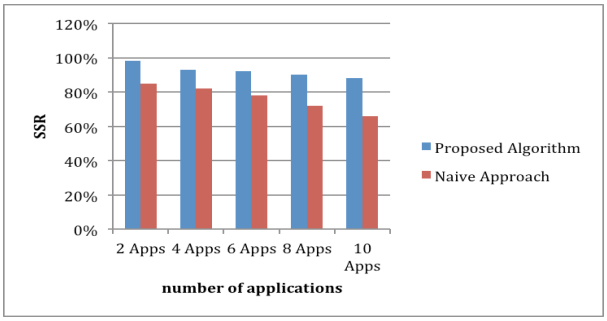


Figure 3. SSR with a different number of applications

F. Comparison with Greedy and Random Task Allocation Schemes

We compare the performance of our algorithm with two different approaches: a greedy algorithm proposed in [6] and the random, naive approach previously described. In this evaluation, we assess the capability of the algorithms to adjust the changes in SSAN, such as the system workload is continually increased over time. The strategy adopted in the tests is shown below. After the system starts running, in each interval two new applications are added into the system. A time interval lasts for 30 seconds. Therefore, the following intervals were established: in T1, 2 applications were running; in T2 (30 sec after the system initialization), 4 applications were running; in T3 (60 sec after the system initialization), 6 applications were running; and so on, until T5 (with 10 applications are simultaneously running in the system). Figure 4 shows that as the number of applications increases, the system lifetime of all algorithms are decreased, but in our algorithm as the number of applications increase the decrease in system lifetime is slighter than in the Random algorithm, due to the fact that we are sharing tasks among applications. We also have better results compared to the Greedy algorithm. The differences between this figure and figure 2 were due to the adaptation cost that the system had with the arriving applications.

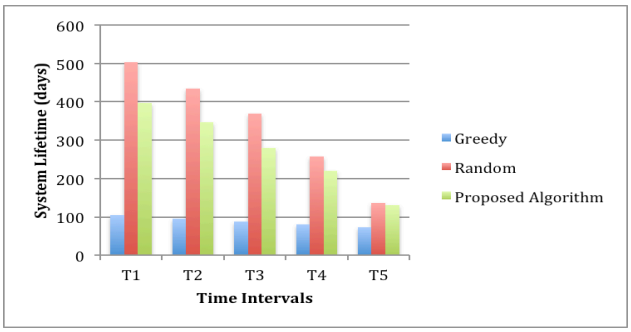


Figure 4. Comparison of system lifetime

In Figure 5, we analyze the differences among the SSR values for the considered intervals and algorithms. The resultant difference denotes the adaptation cost, which means the cost that the algorithms have to schedule the arriving applications. Our algorithm has the best results as the number of applications increase (as we can see in T5, and the same

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DO RIO DE JANEIRO. Downloaded on April 30,2025 at 20:24:08 UTC from IEEE Xplore. Restrictions apply.

results in T3 and T4), followed by the greedy task allocation scheme. As the number of applications increase the SSR of the Greedy algorithm decays faster than the SSR of our algorithm. The proposed algorithm advantage is sharing common tasks, since it will have to schedule just the tasks that were not in the system instead of the complete set of tasks of new applications.

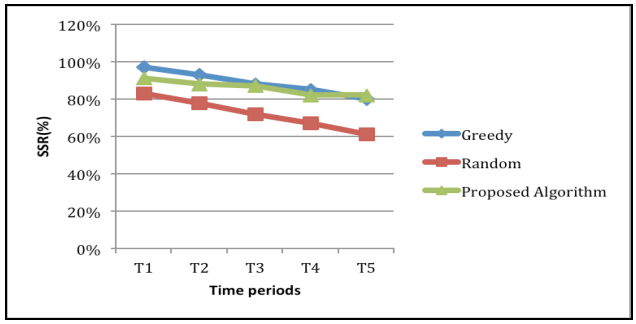


Figure 5. SSR for different time intervals

Finally, we have performed a test to analyze how the packet loss would affect each algorithm performance, since it is well known that wireless communication is very prone to errors and packet loss in real world settings. This was performed changing signal strength using Sun SPOT API. Such test provided very important results (Figure 6): as the packet loss rate increases the SSR of our algorithm decreases. This happens because the proposed algorithm expects useful information to perform the allocation scheme while the random algorithm uses any information at hand and the greedy scheme uses as much as possible. If our algorithm loses information they lose SSR while the greedy algorithm is less sensitive to packet loss. Such results demonstrate that our algorithm has the best performance. The experiments prove that it can effectively adapt to dynamic changes in the number of running applications.

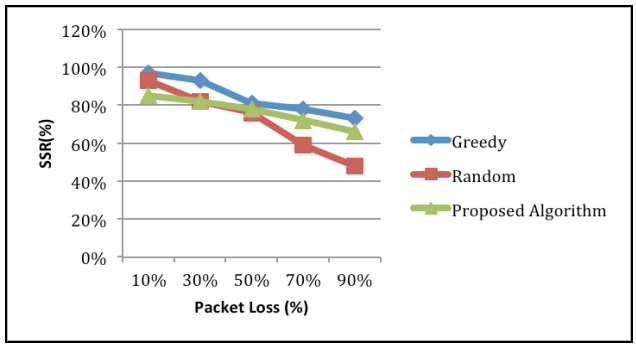


Figure 6. SSR with different leves of packet loss

VI. CONCLUSIONS

In this paper, we address task allocation of multiple applications running on top of a SSAN. We developed a new scheduling algorithm that exploits common tasks to make effective task-sensor assignments explicitly take full advantage of applications with common tasks and avoid repeating tasks unnecessarily. Experimental results show that our algorithm produces promising results in terms of SSR, energy consumption and system lifetime.

In future work, we will extend the algorithm to work at distributed manner. In such case, each node will have to contribute to evaluate the SSAN knowing that each task contributes with a different QoS values to the application and also consumes different amount of network resources. Our algorithm would need to decide which tasks have to be completed to meet the minimum requirements of applications, while maximizing the system utility for all the applications without using a centralized unity to make decisions.

REFERENCES

[1] Ting Zhu; Mohaisen, A.; Yi Ping; Towsley, D.; , "DEOS: Dynamic energy-oriented scheduling for sustainable wireless sensor networks," INFOCOM, 2012 Proceedings IEEE , vol., no., pp.2363-2371, 25-30 March 2012

[2] Delicato, F. C. ; Protti, F. ; Rezende, J. F. ; Pirmez, L. . An Efficient Heuristic for Selecting Active Nodes in Wireless Sensor Networks. Computer Networks, v. 50, p. 3701-3720, 2006.

[3] S. Xiong, J. Li, M. Li, J. Wang, and Y. Liu, "Multiple Task Scheduling for Low-Duty-Cycled Wireless Sensor Networks," in INFOCOM '11.

[4] Wang, J. et al. (2004) "A sensor-fusion approach for meeting detection" In: Workshop on Context Awareness at Int. Conf. on Mobile Systems, App., and Services (MobiSys), Boston, June 2004.

[5] Efstratiou, C.; Leontiadis, I.; Mascolo, C.; and Crowcroft, J. 2010. A shared sensor network infrastructure. In Proceedings of the 8th ACM Conference on Embedded Networked Sensor Systems (SenSys '10). ACM, New York, NY, USA, 367-368. DOI=10.1145/1869983.1870026

[6] Xu, Y., Saifullah, A., Chen, Y., & Lu, C. (n.d.). Near Optimal Multi-Application Allocation in Shared Sensor Networks. Network.

[7] Bhattacharya, S., Saifullah, A., Lu, C., & Roman, G.-C. (2010). Multi-Application Deployment in Shared Sensor Networks Based on Quality of Monitoring. 2010 16th IEEE Real-Time and Embedded Technology and Applications Symposium, 259-268. Ieee. doi:10.1109/RTAS.2010.20

[8] Leontiadis, I., Efstratiou, C., Mascolo, C. and Crowcrof, Jt. 2012. SenShare: transforming sensor networks into multi-application sensing infrastructures. In Proceedings of the 9th European conference on Wireless Sensor Networks (EWSN'12), Gian Pietro Picco and Wendi Heinzelman (Eds.). Springer-Verlag, Berlin, Heidelberg, 65-81.

[9] Yuan Tian; Ekici, E.; Ozguner, F.; , "Energy-constrained task mapping and scheduling in wireless sensor networks," Mobile Adhoc and Sensor Systems Conference, 2005. IEEE International Conference on , vol., no., pp.8 pp.-218, 7-7 Nov. 2005.

[10] Wilde, E.; Guinard, D.; e Trifa, V. Architecting a Mashable Open World Wide Web of Things, Institute for Pervasive Computing, ETH Zürich, Zürich, Switzerland, No. 663, February 2010.

[11] Li, W., F.C. Delicato, and A.Y. Zomaya, Adaptive energy-efficient scheduling for hierarchical wireless sensor networks. ACM Trans. Sen. Netw., 2013. 9(4).

[12] Paschoalino, R.; Madeira, E.R.M., "A Scalable Link Quality Routing Protocol for Multiradio Wireless Mesh Networks". Proc. of WiMan 2007, Honolulu, 2007.

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DO RIO DE JANEIRO. Downloaded on April 30,2025 at 20:24:08 UTC from IEEE Xplore. Restrictions apply.